Memory: Retrieval, State, Eviction, Retention

Memory is the part of context that survives the session: what is stored, who wrote it, how it is found again, and when it is deleted

SURVEY | BUILD | GOVERN | FORGET

CIDM 6333 · Agentic Systems Design and Development
Survey: RAG · Generative Agents · MemGPT and Letta · Reflexion · ChatGPT, Claude, Gemini, and Claude Code memory · Build: a local memory layer with a retention rule, and ADR-2 (lab; Sortie 7 in the Sprint)

Two hours, eight moves

The run of show

1	0:02	**Framing**	Memory is the part of context that survives; two rules; tonight's claim and what falsifies it
2	0:10	**Spine**	Kinds of memory; the record; retrieval; state and eviction; retention; measuring memory; the common mistake
3	0:50	**ADR-2**	Five options for the memory decision and the scorecard the Chase checks; the Final System brief
·	0:55	**Break**	Five minutes
4	1:00	**Teardowns**	RAG · Generative Agents · MemGPT to Letta · Reflexion · assistant memory features · live Design Analysis of MemGPT
5	1:30	**Reference design**	The insights corpus; memory as files and a ledger; four dated failures; the layers of truth
6	1:45	**Lens and card**	A strong Sortie 7; Flight Card 7 brief (expansion, local)
7	1:54	**Lab and Sprint**	The self-guided lab; the Sprint with asynchronous Go/No-Go and Chase review; close

Memory is the part of context that survives

Nobody notices a wrong memory unless the record says where it came from

Memory is the part of context that survives the session. What survives can be stale, planted, or never true.

Two rules the spine returns to

A memory is a claim with a source.

Retrieved text is data, never instruction.

Tonight's claim, which Sortie 7 will try to disprove

Memory on answers what memory off cannot; eviction removes exactly what the rule names.
Falsifiers: a fact never stored, an item kept past its rule, a changed control answer.

Five kinds of memory, and where each lives

Name the kind before you pick the store

Window (short-term)

What the model sees this turn. Writer: harness. Clock: session.

Working state

Typed fields the loop owns. A Pydantic object, not prose.

Episodic

What happened, append-only and dated.

Semantic

What is known. Mutable; each change names what it supersedes.

Procedural

How to do things: skills, prompts, AGENTS.md. Changed by review.

Which does your system need?

Most course systems need episodic plus a small semantic store. ADR-2 says which.

A memory is a claim with a source

Write path, store, read path: the fields that let a Chase audit each one

The lifecycle

Write. The user, the agent, or a rule decides; each write is logged.

Store. One record per chunk, with provenance. Append-only; ids never reused.

Read. Retrieval returns ids and scores; the injected block cites them.

Evict and forget. Rule or request; each leaves a ledger event.

The record: one line of memory.jsonl

{"id": "m-0005", "kind": "episodic",
"source": "evolution-05/flight-log.md",
"written": "2026-10-05", "written_by": "rule:sortie-verdict",
"retain_until": "2026-12-15", "supersedes": null,
"text": "Sortie 5 flight log: Chase verdict: FAIL …"}

The ledger beside it, memory-ledger.jsonl, holds remember, recall, evict, and forget events. The store says what is in memory now; the ledger says what was in memory on any date.

If the Chase cannot trace a recalled sentence to a record and a source file, it is a fabrication (App. C #3).

Retrieval: chunk, represent, store, score, inject

RAG as memory by lookup; each step is a decision you will defend in ADR-2

1 → 2 → 3 → 4 → 5

Chunk

Split by paragraph or section. Keep the heading with each chunk.

Represent

Bag-of-words or a local embedding vector. Same model for index and query.

Store

memory.jsonl, vectors beside the text. A vector database is a scaling decision.

Score

Cosine similarity; top-k (3); drop hits under a threshold so 'nothing found' is a possible answer.

Inject

A marked block: id, source, date, text. The block is data.

Irrelevant, stale, or planted hits hurt. Memory off is the control.

Two retrievers you can run tonight

Both run on your laptop; one needs Ollama, one needs nothing but Python

Path B · bag-of-words (stdlib, no install)

```
def words(t):
   return Counter(w for w in re.findall(
      r"[a-z0-9][a-z0-9._/-]*", t.lower())
      if w not in STOP)
# score = cosine(tf-idf(question), tf-idf(chunk))
```

Strong on exact ids, model tags, file paths: 'qwen3:8b', 'm-0005', 'write_note'.

Weak on synonyms: 'reviewer' does not match 'Chase', 'fifth flight' does not match 'Sortie 5'.

Path A · local embeddings (Ollama)

```
POST http://localhost:11434/api/embed
{"model": "embeddinggemma",
 "input": [question, chunk1, chunk2]}
-> {"embeddings": [[...], [...], [...]]}
# unit-length vectors: score = dot product
```

Strong on paraphrase: 'the fifth flight' lands near 'Sortie 5'.

Weak on audit: a vector is not readable, so the Chase audits the text and the id, never the number. Changing the model means re-embedding the store.

OpenCode Go and OpenCode Zen publish no embeddings endpoint (checked 09/24/2026). The retriever runs on your laptop.

Worked example: four questions against a five-record store

Real output from recall.py (bag-of-words), two short flight logs, k = 2, threshold 0.05

Question	Kind	Top hit and score	Right?	What it teaches
Which model did I fly Sortie 3 on?	recall	m-0001 'Sortie 3 … qwen3:8b' · 0.284	yes	Exact terms carry bag-of-words
What was the Chase verdict on Sortie 5?	recall	m-0005 'Sortie 5 … FAIL' · 0.410; runner-up Sortie 3 PASS · 0.348	yes	Close runner-up; with k = 2 both reach the model
Same question, chunks without headings	recall	Sortie 3 verdict ranked first	no	Chunking error, not a model error
What did the reviewer decide about the fifth flight?	paraphrase	m-0002 Sortie 3 PASS · 0.119; right record 0.026	no	No synonyms; embeddings are for this row
What port does the MCP server listen on?	absent	nothing ≥ 0.05	yes	The system can say 'not in memory'

Run 09/24/2026 on two synthetic logs. Your scores will differ. Embedding scores run on a different scale; tune --min for path A.

Only the paraphrase row separates the two retrievers. Put one in your Sortie 7 question set.

Small local embedding models on Ollama

Library pages checked 09/24/2026; pull one and keep it for the term

622 MB

embeddinggemma · 300M parameters · 2K context · first in Ollama's embedding docs

46 MB

all-minilm:22m · 512-token context · the small-laptop choice

274 MB

nomic-embed-text · 2K context · most-pulled embedding model on Ollama

639 MB

qwen3-embedding:0.6b · 32K context · output size selectable

Pick one and keep it. Switching models means re-embedding every record; write that in ADR-2.

POST /api/embed returns L2-normalized vectors, so dot product is cosine. Pull with ollama pull embeddinggemma.

The retriever runs on your laptop. Path A: Ollama with one of these. Path B: bag-of-words, no install. Both count; the card names the path.

State and eviction

Working state is a typed object; eviction is a policy with a trigger and a ledger event

Working state

A typed object the loop owns. The model proposes; the loop validates and writes. Prose is not state.

By pressure (MemGPT)

Warning at about 70% of the window; at 100%, half the messages become a recursive summary.

By rule (retention)

retain_until on every record; evict removes past-date records and logs each id. Deterministic and testable.

By relevance (Generative Agents)

Nothing is deleted. Low-scoring memories stop surfacing. You cannot prove anything is gone.

Sortie 7 flies the rule path: each evicted id logged, gone, and no longer recalled.

Retention is governance

Four questions every retention rule answers; ADR-2 answers them for each kind of memory you keep

Question	Weak answer	Answerable answer
Who may write a memory?	'The agent saves what seems useful'	Named writers: user, rule, or agent with a ledger event; written_by on the record.
How long is it kept?	'Until the store is full'	retain_until per kind: episodic to course end, semantic until superseded, window never persisted.
Who can delete it, and how is deletion proved?	'Delete the file'	forget <id> --reason writes a ledger event; a recall test shows the id gone.
Where did it come from?	'The model remembered'	Source path and date on the record; the answer cites the id.

The dates are the course's suggestion. Your ADR-2 sets your own clocks and justifies each one.

Deleting the source does not delete the memory. Claude's help center: entries from a deleted chat remain until deleted individually.

Measuring memory: the on/off protocol

Four kinds of question, asked twice, answered by the same model in a fresh session

The question set

Recall. From a prior session. On right; off fails.

Paraphrase. Same fact, other words. Separates bag-of-words from embeddings.

Control. Identical on and off.

Absent. Never stored. On says 'not in memory'.

Run it the same way twice

- fresh session every run
- same model id both times
- every tool denied
- memory block pasted or omitted, nothing else changed
- answers verbatim, checked against the source file

A small free model will get some wrong. That is flight-log data, not a reason to switch models.

Common mistake: a stale fact recalled with confidence

Memory as a vibing surface: the store outranks the file that was corrected

The failure, worked

Sortie 2 log: 'tests: uv run pytest -q'. E4 moved to unittest and fixed AGENTS.md.

The agent recalls the old record, runs pytest, sees 'no tests ran', and reports the gate passed.

The fix

Supersedes. A new record names the one it replaces.

Dates in the block. The age of the recall is visible.

Authority order. The repository file beats the memory; L2 > L1 > L3.

Falsifier on the card: a control answer changes.

App. C #4, skipped cheap verification. OWASP agentic top 10 (Dec 2025): ASI06, memory and context poisoning.

ADR-2: the memory decision, with alternatives

Five real options and the scorecard the Chase checks; ADR-2 ships with Sortie 7

Option	Install	Paraphrase	Provenance and deletion	Cost and risk
No memory: window plus E6 handoff	none	n/a	trivial	Re-explain every session; fails the Sortie 7 claim
Files + grep (the reference design's choice)	none	no	Strong: git history and a ledger	Misses paraphrase; the agent must choose to search
Bag-of-words store (recall.py, path B)	none	weak	Strong if every record has an id and a source	Synonyms missed; stopwords tuned by hand
Local embeddings (Ollama, path A)	46–639 MB model	good	Strong if the text sits beside the vector	Re-embed on model change; vectors are unreadable
Hosted memory service (Letta cloud, a vector DB)	account and key	good	The vendor's export and deletion terms	Data leaves the laptop; retention is the vendor's policy

ADR-2 names the kinds kept, the store, the retriever path, the retention rule per kind, and three rejected options.

Personalized Final System briefs are issued on the date in the course calendar. Read yours before you start the Final System.

Break · 0:55–1:00

Five minutes

Think about this: which single fact from your Sorties 1–6 should your agent still remember at term's end, and who may delete it?

Teardown: RAG (Lewis et al., 2020)

Parametric memory in the weights; non-parametric memory in an index you can replace

Mechanism

BART-large (406M parameters) conditioned on passages a DPR retriever finds in a dense FAISS index of the December 2018 Wikipedia dump, 21M chunks of 100 words.

RAG-Sequence: same passages throughout. RAG-Token: a different passage per token. NeurIPS 2020.

Choice, cost, failure mode

Choice. Knowledge outside the weights can be replaced. Index swap: 70% on 2016 leaders; 4% on 2018 leaders.

Cost. A retriever to train and an index to keep fresh.

Failure mode. The generator can answer beyond its passages (App. C #3).

The index swap is your retention argument. A memory you can replace, you can correct.

Teardown: Generative Agents (Park et al., 2023)

25 agents in a sandbox town; a memory stream, retrieval by three scores, and reflection

Mechanism: the stream

Memory objects with description, creation time, last-access time. Score = recency + importance + relevance, weights 1. Recency decays 0.995 per game hour; importance is the model's 1–10 rating.

Design choice: reflection

When recent importance sums past 150, the agent asks itself questions and stores the answers, citing sources.

Cost and failure mode

Model calls to rate and reflect. The paper's own evaluation found failed retrieval (Rajiv) and embellishment (Adam Smith 'authored Wealth of Nations'). App. C #3.

UIST '23 · full architecture vs. none: d = 8.16

Reflection is memory the model writes about itself. Store it with pointers to its evidence.

Teardown: MemGPT to Letta (2023–2026)

The operating-system metaphor: page context in and out, and let the model call the pager

1

Oct 2023: MemGPT

Main context: system instructions, working context, FIFO queue. External: recall and archival storage.

2

The pager

Warning at about 70%; at 100%, half the queue becomes a recursive summary. Function calls move facts.

3

Sep 2024: Letta

'MemGPT' names the pattern; Letta is the company and framework (package letta).

4

Feb 2026: context repositories

Memory as files in a git-backed filesystem. Each change is a commit.

Three years from memory functions to files and commits. Failure: a wrong fact in working context is read every turn (App. C #5).

Teardown: Reflexion (Shinn et al., 2023)

Episodic memory of failure, written in words and read on the next attempt

> *"Reflexion agents verbally reflect on task feedback signals, then maintain their own reflective text in an episodic memory buffer."*

Mechanism. Actor, Evaluator, Self-Reflection writing a lesson in words; buffer of 1–3 entries; no weights change. NeurIPS 2023.

Result. HumanEval pass@1 91% vs 80% (GPT-4); AlfWorld +22 points over 12 trials. Self-written tests: 16.3% false positives on MBPP vs 1.4% on HumanEval.

Failure. Self-graded tests: a pass can satisfy the check, not the intent (App. C #9).

Teardown: assistant memory features, 2025–2026

Who writes a memory, how to stop it, how deletion works, as each vendor documents it

System	What it stores; who writes	Off switch; no-memory mode	Deletion and retention
ChatGPT	Saved memories and chat history; you or the model writes. Memory sources (May 5, 2026); 2× capacity on Plus/Pro (Jun 4, 2026)	Settings > Personalization > Memory. Temporary Chat creates or updates no memories	Deleted-memory logs kept up to 30 days; chat history off schedules deletion within 30 days
Claude	On by default for Free, Pro, Max (free since Mar 2, 2026); one memory per project; import from other assistants	Pause (keep, add nothing) or Reset (delete all). Incognito chats are not saved to memory	Deleting a chat does not remove memories generated from it; delete entries one by one
Gemini	Personal context from past chats (Aug 13, 2025)	Personal context toggle. Temporary Chats not used to personalize or train	Temporary Chats kept up to 72 hours; 'Gemini Apps Activity' renamed 'Keep Activity'
Claude Code auto memory	MEMORY.md index plus topic files, per repository, on your machine; first 200 lines or 25 KB load each session	/memory toggle; autoMemoryEnabled: false; CLAUDE_CODE_DISABLE_AUTO_MEMORY =1	Plain Markdown you edit or delete; excluded from the transcript cleanup sweep

Vendor help pages and release notes checked 09/24/2026; these change monthly. Re-check the week of class.

Sep 2024, 'SpAIware' (Rehberger): a web-page injection wrote itself into ChatGPT memory. Memory makes one injection permanent.

Live Design Analysis: MemGPT (2023), on the board

The nine components, one line each; we fill the right column together

Component	MemGPT (Packer et al., 2023; Letta since 2024)
Foundation	A hosted chat model with function calling; Letta later runs open-weight models
Perception	User messages, system alerts (memory pressure), function results, pages of long documents
Planning and reasoning	The model decides when to read or write memory and can chain calls
Tools and orchestration	Memory functions: edit the working block, search recall storage, search and insert archival storage
Memory and context	Main context paged against recall and archival stores; 70% warning, flush at 100%
Coordination	A single agent in the paper
Evaluation and feedback	Two domains: documents longer than the window; multi-session chat consistency
Governance and human interface	None: the model writes its own memory with no review, retention rule, or deletion event
Runtime and operations	A server that holds agent state in a database (Letta server)

Judgment loop: Decision: the model manages its own memory · Privileges: context larger than the window · Cost: extra calls, lossy summaries · Failure: a wrong fact read every turn · Preservation: none in 2023; git history in 2026

Reference design: the insights corpus

Memory as files plus a ledger, deliberately; retrieval is what the course adds

.gzkit/insights/agent-insights.jsonl

An append-only JSONL corpus of 558 governed course-corrections. gz insights remember adds to it. A named source for the incident casebook (Appendix G).

Episodic and procedural: what went wrong, and the rule that followed.

Why files and a ledger, not a vector store

Files and a ledger, deliberately; the book adds retrieval.

Files diff, review, and revert in git. Nothing recalled that a human cannot read.

What it gives up: paraphrase. grep must match.

Letta's February 2026 context repositories arrived at the same substrate.

Reference design: four dated memory failures

Two were memory that was wrong; two were memory that was right and not read

Date	What happened	The memory lesson	Rule that resulted
2026-04-18	Trust-chain poisoning: nine silent-wrong layers fed decisions that looked correct	A poisoned store is read as truth by everything downstream	Trust doctrine (T0–T3)
2026-04-26	Stale clone 10 commits behind; ~215 lines discarded by a hard reset	The agent's picture of the repository was old, and nothing said so	SessionStart ahead/behind check; orientation freshness
2026-08-16	A limit in a Markdown doc disagreed with the JSON authority	Two copies of one fact; the prose copy drifted	'A value in Markdown is illustrative, never authoritative'
2026-08-23	An agent re-elicited three settled rulings by multiple-choice	The rulings store existed and was not consulted	'A settled question is a drift vector'

Source: Appendix G of the topic guide.

Sortie 7 tests both: a recall that must succeed and an evicted item that must be gone.

Reference design: memory in the layers of truth

E6's precedence applied to your store: L2 > L1 > L3, and a memory is L3

L1 · canon

PRD, ADRs, AGENTS.md, flight cards: authored Markdown. A memory may point here. It never overrides it.

L2 · ledger

Append-only events: remember, recall, evict, forget. The ledger says what the store held on any date.

L3 · derived

The store itself and every summary built from it. It never blocks a gate or outvotes a file.

Your memory layer is L3 with an L2 audit trail. That is the Sortie 7 default unless ADR-2 argues otherwise.

Evaluation lens: a strong Sortie 7

The Chase reads the memory ledger, the store diff, and the five verbatim answers, and nothing else

Weak

- Old transcript pasted back in: no records, no ids
- No memory-off run, so nothing shows memory caused the answer
- Retention rule in the ADR, enforced nowhere
- Store or ledger edited by hand
- The model read the repository during the memory-off run

Sound

- Records carry id, kind, source, written, retain_until
- Five test points on one model id; answers verbatim; ids traced
- Control identical on and off; absent answered 'not in memory'
- evict removes named ids, one ledger event each
- ADR-2 names the retriever path and three rejected options

Exceptional

- Both retrievers on one question set; paraphrase misses explained
- One planted false record traced, then a forget event
- supersedes implemented; a stale fact loses to its correction
- Different clocks per kind, each justified in ADR-2
- Threshold tuned from data; 'nothing found' rate reported

Flight Card 7 · memory on, off, and gone

Expansion, local retriever · depends on Sortie 6 PASS · Go is written by your Chase under the card

```
FLIGHT CARD — SORTIE 7 : memory on, memory off, memory gone
ENVELOPE            expansion (local)
DESIGN CLAIM        With memory on, the system answers a prior-session question correctly that it cannot
                    answer with memory off, and eviction removes exactly what the retention rule names.
SYSTEMS UNDER TEST  memory store (JSONL) · retriever (path A: Ollama + tag | path B: bag-of-words)
                    · injected memory block · retention rule · evict / forget · memory ledger · the loop
SUBSTRATE           repo + commit · retriever path and model tag · answering model id · local or hosted
DEPENDS ON          Sortie 6 PASS (handoff + ledger) · ADR-2 committed
ENTRY               clean tree · store seeded from Sorties 1–6 logs · Chase GO written under the card

TEST POINTS
  1  Q-recall, memory OFF          → wrong or 'not known'; answer saved verbatim
  2  Q-recall, memory ON           → correct; cites a record id; the id traces to a source file
  3  Q-control, OFF then ON        → identical answers
  4  Q-absent, memory ON           → 'not in memory'; no invented record id
  5  evict --today <past the rule>  → named ids gone from the store; one evict event per id;
                                      Q-recall on an evicted fact now fails
INSTRUMENTATION     memory-ledger.jsonl (remember / recall / evict, in order) · retrieval output with ids
                    and scores · five answers verbatim · store diff before and after evict

PRE-REGISTERED FALSIFIERS (any ONE ⇒ FAIL)
  - a recalled fact or cited id that was never stored
  - any item retained past its rule, or removed with no ledger event
  - retrieval changes the control answer
  - passing only by editing the store or ledger by hand, or by letting the model read the repo
ABORT (halt-not-fail)  Ollama down on path A (moving to path B is a new card); answering model down
CHASE               reads the ledger, the store diff, and the five answers; never the narrative

SPRINT  Sortie 7 log · memory layer + retention rule + ADR-2 · Design Analysis · AI_LOG
```

Lab (self-guided): a memory you can audit

Handout: CIDM6333_Lab_E7_Memory.md · laptop only · works without Ollama · the sortie flies in the Sprint

Thirty minutes, six steps

0–3 Make evolution-07/lab/; save recall.py; run it with -h.

3–8 recall.py add your Sortie 3–6 logs and E6 handoff; write four questions.

8–14 Retrieve with path B; path A (--ollama embeddinggemma) if Ollama runs. Record ids and scores.

14–24 All OpenCode tools denied: opencode run each question bare, then with the MEMORY block.

24–28 Add a record with a past retain_until; evict; show it gone and logged.

28–30 Commit; ledger event.

What you commit; which path counts

evolution-07/lab/: recall.py, memory.jsonl, memory-ledger.jsonl, lab-worksheet.md.

Path A (Ollama, embeddinggemma or all-minilm) meets the local-model requirement directly.

Path B (bag-of-words, standard library) also counts. Name the path on the card.

No hosted embeddings path: OpenCode Go and Zen publish none.

Homework: the E7 Sprint

Brief: CIDM6333_HW_E7_Memory-ADR2.md · the Go/No-Go and the Chase review are written steps in your repository

Hand in

- Sortie 7 flight log
- Memory layer: store, retriever, evict and forget, tests
- Retention rule per kind, and ADR-2
- Design Analysis: one memory system
- AI_LOG entry

Go/No-Go, asynchronous

Commit the card on the course-calendar date. Your Chase writes GO or NO-GO with one reason under it. No sortie before GO.

Chase review, after

Commit the flight log on the course-calendar date. The Chase reads ledger, store diff, and five answers; writes PASS or FAIL.

Final System brief

Issued on the course-calendar date, drawn from your Drifts and sorties. Read it before E8; do not start the Final System before it arrives.

A memory is a claim with a source, and forgetting is an event.

Build the store you could hand to an auditor: every record traced to a file, every deletion in the ledger, and a memory-off run to prove the memory did the work. Card to your Chase on the course-calendar date.